

D3D12 延遲著色渲染器 — 實驗報告

NTUST 計算機圖學 — Assignment 1: Deferred Shading

項目	內容
平台	Windows 11 / Visual Studio 2022 / DirectX 12
圖形卡	NVIDIA GeForce RTX 3060 (12 GB)
作者	Claude (Anthropic) · 與專案擁有者於同一次互動式開發中協作完成

關於本份報告。本專案的所有 C++ / HLSL 原始碼、Visual Studio 專案設定、vcpkg manifest 以及這份報告的內容皆由 Claude (Anthropic) 撰寫。專案擁有者提供開發環境、模型資產、與逐步的實作方向；API 設計、資料結構、Shader、以及兩階段管線結構的選擇皆出自我的判斷。

1. 概述

本作業實作了一條完整的兩階段「Deferred Shading」渲染管線，使用 DirectX 12 API 從零搭建。場景為 Crytek Sponza，透過 Assimp 載入後在 Geometry Pass 寫入 G-buffer，再於 Lighting Pass 以一個全螢幕三角形 (Full-Screen Triangle) 套用 Blinn-Phong 平行光照模型輸出最終影像。

程式啟動時會同時開啟 Console 與 render window 兩個視窗：Console 接受文字指令，render window 接受鍵盤與滑鼠操作。執行完整的 view-mode 切換測試 (Depth → Normal → Albedo → Color → quit) 過程中 D3D12 Debug Layer 沒有任何 ERROR 等級訊息，符合作業 §7.1 的嚴格要求。

2. 實作方式

2.1 程式架構

程式碼依職責切割為幾個聚焦的翻譯單元：

檔案	負責
Main.cpp	程式進入點、Win32 視窗類別、Message Pump、Console 執行緒、指令分派
Renderer.cpp / .h	D3D12 Device 與 Swap Chain、兩條 PSO、兩階段 Render
Camera.cpp / .h	第一人稱 yaw / pitch 相機、XMMatrixPerspectiveFovLH
Scene.cpp / .h	VB / IB / Submesh 表 / 材質 SRV Heap / 場景包圍盒
AssetLoader.cpp / .h	Assimp 場景拜訪、頂點聚合、DDS 載入 (透過 DirectXTK12)
Console.cpp / .h	背景 std::cin 執行緒、Mutex 保護的指令佇列
Input.h	鍵盤、滑鼠 Delta、Look-Mode 旗標的共享狀態

連結器使用 /SUBSYSTEM:CONSOLE，stdout / stderr / stdin 由 Loader 自動接好；render window 透過 CreateWindowExW 額外建立。Console 指令在背景執行緒以 std::getline 讀入並推進 mutex 保護的佇列，主執行緒在每幀 Render 前消化佇列，std::getline 因此不會阻塞 Render Loop。

2.2 資產載入

Assimp 以下列 Post-Process Flag 載入 sponza.obj：

```
aiProcess_Triangulate
| aiProcess_GenSmoothNormals
| aiProcess_ConvertToLeftHanded
| aiProcess_PreTransformVertices
| aiProcess_JoinIdenticalVertices
| aiProcess_ImproveCacheLocality
```

PreTransformVertices 把整個 Node Hierarchy 攤平、Transform 焙進頂點位置，因此 Renderer 可以把整個場景視為已在 World Space、Model Matrix 為

identity。ConvertToLeftHanded 是 MakeLeftHanded + FlipUVs + FlipWindingOrder 的組合，把 OBJ 轉成 D3D 慣用的左手座標。

解析後的 Sponza 統計：

項目	數值
頂點數	193,372
索引數	837,489
Submesh	26 個
材質貼圖	27 張 (含 1×1 白色 fallback)
場景包圍盒 (X / Y / Z)	(-1920.9 .. 1799.9) / (-126.4 .. 1429.4) / (-1105.4 .. 1182.8)

頂點格式採用作業要求的 Position (R32G32B32_FLOAT) | Normal (R32G32B32_FLOAT) | UV (R32G32_FLOAT) · Stride 32 bytes。

Sponza 的 .mtl 雖然引用 .tga，但同目錄下也提供了對應的 .dds。Loader 在解析每個材質時會把副檔名改成 .dds / .DDS 嘗試，皆失敗才退回原始檔名。所有貼圖透過 DirectX12 的 ResourceUploadBatch 與 CreateDDSTextureFromFile 上載；上傳完成後立刻 Transition 到 PIXEL_SHADER_RESOURCE。沒有 Diffuse 貼圖的材質則 fallback 到一張 1×1 白色貼圖。所有 SRV 集中在一個由 Scene 持有的 Shader-Visible CBV/SRV/UAV Heap，每個 Submesh 紀錄自己對應的 SRV Index，Geometry Pass 在 Draw 之前以 SetGraphicsRootDescriptorTable 切換到該材質。

2.3 Phase 1 — Forward Rendering (中介驗證)

依作業 §4 的建議，在拆出 Deferred Pipeline 之前先以一條最單純的 Forward 路徑驗證：Assimp 載入是否正確、頂點 / UV / 法線是否傳對、貼圖是否綁對、Blinn-Phong 是否能在 Forward 上正確著色。Forward 階段使用一個 Forward.hlsl 直接寫到 Swap Chain 後緩衝；當 Sponza 在 Forward 上能正確顯示後，Forward.hlsl 即被刪除，由 Geometry.hlsl 與 Lighting.hlsl 取代，繳交版的原始碼不再含 Forward Shader。

2.4 Phase 2 — Deferred Shading

2.4.1 G-buffer 配置

三條 G-buffer 的格式完全依作業 §3.B 的規格：

Buffer	Resource Format	View 型別	內容
Depth	R32_TYPELESS	DSV: D32_FLOAT SRV: R32_FLOAT	硬體深度，於 Lighting Pass 取樣
Normal	R16G16B16A16_FLOAT		世界空間法線於 xyz；w 固定為 1.0
Albedo	R8G8B8A8_UNORM		取樣後的 Diffuse RGB；w 固定為 1.0

Depth Buffer 必須能同時被 DSV 寫入與 SRV 讀取，所以底層 Resource 用 R32_TYPELESS + ALLOW_DEPTH_STENCIL，再分別建立兩個型別不同的 View。

RTV Heap 大小為 kFrameCount + 2：Index 0–1 是兩個 Swap Chain Back Buffer、Index 2 是 Normal RT、Index 3 是 Albedo RT。Lighting Pass 用的 G-buffer SRV 則放在另一個 3 槽的 Shader-Visible Heap，順序固定為 { depth, normal, albedo }，整個以單一個 Descriptor Table 一次綁完。

2.4.2 Geometry Pass — Geometry.hlsl

Vertex Shader 把 Position 經 viewProj 變換到 Clip Space，把 Normal 與 UV 直接送給 Pixel Shader。Pixel Shader 寫兩個 MRT：

```
struct PSOut {  
    float4 normal : SV_TARGET0;    // RGBA16F  
    float4 albedo : SV_TARGET1;    // RGBA8  
};
```

Geometry Pass 的 Root Signature：

Slot	類型	用途
------	----	----

0	Root CBV (b0)	PerFrame 常數 (viewProj、camera、light、viewMode、...)
1	Descriptor Table → SRV (t0)	當前 Submesh 的 Diffuse 貼圖
—	Static Sampler (s0)	Linear-Wrap (Min/Mag/Mip Linear, Address Wrap)

Depth State 為 LESS、Depth Write 全開。

2.4.3 Lighting Pass — `Lighting.hlsl`

Lighting Pass 使用真正的「全螢幕三角形」(Full-Screen Triangle)，不是兩三角形組成的四邊形。Vertex Shader 不需要任何 Vertex Buffer，從 `SV_VertexID` 直接合成 NDC 與 UV：

```
o.uv = float2((vid << 1) & 2, vid & 2);
o.pos = float4(o.uv * float2(2.0, -2.0) + float2(-1.0, 1.0), 0.0, 1.0);
```

三次 VS 呼叫即覆蓋整個 viewport，且沒有 Quad 在對角線會出現的接縫 / 重疊著色。Pixel Shader 透過 `Texture2D::Load` (用 `SV_POSITION` 直接取整數像素座標、不需 Sampler) 讀取三個 G-buffer Slot，並依 `viewMode` 分支：

viewMode	畫面內容
0 — Depth	讀 NDC Depth、用 LH-Perspective 反推回 view-space Z (公式見 § 4.4)、再除以 <code>zFar</code> 顯示
1 — Normal	把 G-buffer 的 [-1, 1] 法線重新映射到 [0, 1] (依作業 §3.D 要求)
2 — Albedo	直接輸出 <code>g_albedo.rgb</code>
3 — Color	Blinn-Phong 平行光：環境 + 漫反射 + 鏡面，世界座標由 <code>invViewProj × NDC</code> 反推

平行光參數完全依 §3.C：強度 (1, 1, 1, 1)、Direction-to-Light (-0.577, -0.577, -0.577, 1.0)。Blinn-Phong 的 shininess 取 64、ambient 為 0.10 × albedo。

2.4.4 PerFrame 常數

每幀以 DirectXTK12 的 GraphicsMemory (Rolling Upload-Heap Pool) 配出一塊 176 byte 的 Constant Buffer，同時被兩條 Pass 的 b0 共用：

```
struct PerFrameCB {  
    XMFLOAT4X4 viewProj;          // 64 B  
    XMFLOAT4X4 invViewProj;       // 64 B // 給 Lighting Pass 反推世界座標  
    XMFLOAT3    cameraPos;        UINT viewMode;          // 16 B  
    XMFLOAT3    lightDirToLight;  float zNear;        // 16 B  
    XMFLOAT3    lightIntensity;   float zFar;         // 16 B  
}; // 共 176 byte
```

HLSL 端宣告為 row_major float4x4，與 DirectXMath 的 row-major 記憶體配置直接相容、不需轉置。

2.4.5 Resource State 循環

每幀以兩次批次化的 ResourceBarrier 包夾 Lighting Pass：

時機	Normal RT	Albedo RT	Depth	Back Buffer
進入 Lighting	RT → PSR	RT → PSR	DEPTH_WRITE → PSR	PRESENT → RT
離開 Lighting	PSR → RT	PSR → RT	PSR → DEPTH_WRITE	RT → PRESENT

幀末 G-buffer 與 Depth 回到「下一幀 Geometry Pass 預期的初始狀態」(RT / DEPTH_WRITE)，因此第一幀完全不需要任何特例。

2.5 相機與輸入 (最後採用 Alt 切換式 Look Mode)

第一人稱相機由 yaw / pitch 兩個歐拉角構成，Forward / Right / Up 由球座標公式合成。Pitch 限制在 $\pm\pi/2 - 0.01$ 以避免 Gimbal Flip。預設移動速度 800 unit / sec、按住 Shift 加速 4 倍；Sponza 沿著最長軸約 3700 unit，這個速度足以舒適地穿行整個場景。

滑鼠處理是這次最反覆的部分。最早的版本是「按住 LMB 拖曳旋轉」+「拖曳期間鎖到中心並隱藏 Cursor」，但這個組合在實際使用時體驗很差：拖曳開始 / 結束的瞬間 Cursor 會閃一下、Cursor 滑出視窗範圍時會打斷旋轉、再加上下一節 §4.6 提到的 race condition，操作很容易卡住。

最終版改成 Alt 切換式 Look Mode：

機制	對應實作
進入 / 離開 Look Mode	按一下 Alt 切換 (<code>WM_SYSKEYDOWN + VK_MENU</code> ，過濾自動重複)
抑制 Alt 啟動視窗系統選單	<code>WM_SYSCOMMAND</code> 收到 <code>SC_KEYMENU</code> 直接 return 0
相機 Delta	<code>WM_INPUT</code> Raw HID，取 <code>RAWMOUSE::lLastX / lLastY</code> (與 Cursor 在不在中心無關)
視覺鎖定	<code>WM_MOUSEMOVE</code> 中 <code>SetCursorPos</code> 把 Cursor 拉回中心； <code>ClipCursor</code> 把 Cursor 限制於 client rect
視覺隱藏	<code>ShowCursor(FALSE)</code> 進入 Look、 <code>ShowCursor(TRUE)</code> 離開
復原原位	進入時 <code>GetCursorPos</code> 存起來，離開時 <code>SetCursorPos</code> 還原
失焦保護	<code>WM_KILLFOCUS</code> 強制清除鎖定，避免 Alt-Tab 後 Cursor 仍隱藏

進入 Look Mode 後，滑鼠隨意動就是相機旋轉；按住 RMB + 滑鼠則切到平移 (沿 right / up)。WASD / QE / Shift 在任何模式下都生效。再按一次 Alt 即離開 Look Mode、Cursor 恢復可見並回到原位。

Z 鍵與 Console 的 `view/cycle` 指令都依作業 §3.D 規定的順序循環：Depth → Normal → Albedo → Color → (回到 Depth)。實作上是 `Renderer::CycleView` 內的 `(int(view) + 1) % 4`。

2.6 D3D12 Debug Layer

Debug 組態下同時啟用 D3D12 Debug Layer 與 DXGI Debug Factory Flag，並在 ID3D12InfoQueue 上對 CORRUPTION 與 ERROR 嚴重度設定 break-on，INFO 等級訊息透過 Storage Filter 過濾掉，確保 Console 乾淨可讀。執行完整的 cycle + view + quit 測試序列下，stderr 沒有任何 D3D12 ERROR 訊息。

2.7 建置系統

單一 Visual Studio 2022 (Toolset v143)、C++17、x64-only。重要設定：

項目	設定
輸出位置	build/x64/{Debug,Release}/ — 一鍵 rmdir /s build 即可清乾淨
編譯器旗標	/utf-8 (避免路徑中文字元觸發 CP_950 警告) + /W4 (整份程式 0 warning)
連結器	/SUBSYSTEM:CONSOLE；連結 d3d12.lib;dxgi.lib;dxguid.lib;d3dcompiler.lib;DirectXTK12.lib;assimp-vc143-mt[d].lib
vcpkg	Manifest Mode、Triplet x64-windows、安裝根目錄重定向到 ASCII 路徑 C:\vcpkg-cache\installed
DLL 部署	Post-Link MSBuild Target，把 assimp-vc143-mt[d].dll、DirectXTK12.dll、z[d].dll 等複製到 exe 旁
Shader	HLSL 透過 D3DCompileFromFile 在執行期編譯 (vs_5_1 / ps_5_1)；shader 檔同時被 CopyToOutputDirectory 複製到輸出目錄

3. 操作說明

3.1 Render Window — 鍵盤、滑鼠

輸入	動作
Alt	切換 Look Mode (進入 / 離開) · 進入時 Cursor 隱藏並鎖在視窗中心
W / A / S / D	前後左右移動 (永遠生效)
Q / Ctrl	下降
E / Space	上升
Shift	移動加速 4 倍
滑鼠 (Look Mode 中)	旋轉相機 (yaw / pitch)
RMB + 滑鼠 (Look Mode 中)	沿 right / up 軸平移相機
Z	循環切換 Depth → Normal → Albedo → Color
ESC	結束程式

重要：相機只在 Look Mode (按下 Alt 之後) 才會跟著滑鼠轉動；想結束鎖定再按一次 Alt 即可。沒有 Look Mode 時 Cursor 維持系統正常游標。

3.2 Console — 文字指令

指令	說明
<code>help</code>	列出所有指令

<code>view depth normal albedo color</code>	切到指定 view mode
<code>cycle</code>	等同於按一次 Z 鍵
<code>pos</code>	列印目前相機位置與 yaw / pitch
<code>quit</code> 或 <code>exit</code>	結束程式

4. 困難與解決方案

4.1 vcpkg 與專案路徑中的中文字元

本專案根目錄含有 `工作\課` 兩段中文字。最初的 vcpkg 安裝在 `zlib` 與 `minizip` 上以 `BUILD_FAILED` 失敗、且 `stderr` 完全空白。Trace `config-x64-windows-out.log` 後發現編譯本身是成功的；失敗發生在 Post-Build 的 `pkgconfig` 修整步驟，那一步會呼叫 `msys2` 的 `pkgconf` 來改 `.pc` 檔，但它無法 round-trip 非 ASCII 路徑。

另外發現 Visual Studio 內附的 `vcpkg.exe` 是 Artifacts-Mode-Only (`vcpkg-readonly: true`)，只能消費 Binary Cache，不能從原始碼編譯任何 Port。

解法兩段：

1. 另外從 GitHub Clone 一份 vcpkg 到 `C:\dev\vcpkg`。
2. 把 vcpkg 所有路徑 — `buildtrees`、`packages`、還有 `install root` — 全部重定向到 `C:\vcpkg-cache` 之下的 ASCII 目錄。MSBuild 的 `<VcpkgInstalledDir>` 也指到這個 ASCII 路徑，徹底繞開專案目錄。

4.2 DirectX::GraphicsMemory 與 inline namespace DX12 的 Forward 宣告陷阱

`Renderer.h` 原本在類別內持有 `std::unique_ptr<DirectX::GraphicsMemory>`，並在 Header 用 `namespace DirectX { class GraphicsMemory; }` 來省 include。這在較舊的 DirectX12 上沒問題，但 vcpkg 目前 (2026-03-31) 提供的版本把

GraphicsMemory 放在 DirectX 內的 inline namespace DX12 { ... } 裡。Inline Namespace 提供的查找穿透性 不適用於「直接在外層 namespace 寫的 Forward 宣告」，所以 Forward 宣告反而引入了一個 不同的、不完整的 DirectX::GraphicsMemory 型別，導致 unique_ptr 上所有成員存取都報「使用未定義類型」。

解法：在 Header 直接 #include <directxtk12/GraphicsMemory.h>，編譯時間略增，但消除了脆弱性。

4.3 Depth 同時當作 DSV 與 SRV

D3D12 不允許同一個 D32_FLOAT Resource 同時被 DSV 與 SRV 綁定。標準解法是底層用 Typeless Format、再依需要建立兩個型別不同的 View。實作上：Resource 用 R32_TYPELESS + ALLOW_DEPTH_STENCIL；Geometry Pass 綁 D32_FLOAT DSV，Lighting Pass 綁 R32_FLOAT SRV。每幀的狀態切換 DEPTH_WRITE → PIXEL_SHADER_RESOURCE → DEPTH_WRITE 確保讀寫永不重疊。

4.4 Depth 視覺化要看得出層次

NDC Depth 直接拿來顯示時幾乎全白，因為 LH-Perspective 的非線性分布會把絕大部分像素的 z 推到接近 1。Lighting Pass 在 PS 內做線性化：

```
zView = (zNear * zFar) / (zFar - zNdc * (zFar - zNear));  
return saturate(zView / zFar).xxx;
```

這樣 Depth 截圖在 Sponza 室內就能看到漸層。

4.5 Lighting Pass 的世界座標重建

計算 Specular 的 $V = \text{camera} - \text{worldPos}$ 需要該 Pixel 的世界座標。为了不浪費一張 G-buffer 在 Position 上 (作業也只規定三張 G-buffer)，改在 Lighting Pass 用 Depth 反推：

```
float4 ndc = float4(uv * float2(2,-2) + float2(-1,1), zNdc, 1);  
float4 world = mul(ndc, invViewProj);  
return world.xyz / world.w;
```

CB 多帶一個 invViewProj，CPU 端每幀算一次 XMMatrixInverse，幾乎沒有額外成本。

4.6 滑鼠：從 LMB 拖曳到 Alt 切換的兩次重做

第一版的相機操作是「按住 LMB 拖曳旋轉」，Delta 直接從 WM_MOUSEMOVE 的座標差取得。當為了「拖曳期間鎖在中心」加上 SetCursorPos 後，SetCursorPos 與 WM_MOUSEMOVE 之間出現 race：Windows 已經排隊但還沒派發的 WM_MOUSEMOVE 仍然帶著舊的絕對座標，造成 Delta 雙倍計算、相機畫面抖動。

第二版把「相機 Delta」與「Cursor 視覺位置」拆開：相機 Delta 改用 WM_INPUT Raw Input，與 Cursor 位置完全無關；WM_MOUSEMOVE 只負責 SetCursorPos 把 Cursor 拉回中心，加上 ClipCursor 限制邊界；WM_KILLFOCUS 失焦時強制清除鎖定。理論上這一版已經正確，但 LMB 拖曳「按下 → 拖 → 放開」這個流程在實際使用上 Cursor 顯示 / 隱藏的瞬間還是會閃，且滑鼠出視窗就斷掉。

最終版乾脆改成 **Alt 切換式 Look Mode**：按一次 Alt 進入鎖定狀態，相機就持續跟著滑鼠 (Raw Input HID)，再按一次 Alt 才解除。LMB 不再參與相機鎖定 (RMB 在 Look Mode 中保留為平移用途)。實作上要注意三件事：

1. Alt 屬於系統鍵，按下會走 WM_SYSKEYDOWN 而不是 WM_KEYDOWN，需要另開一個 case。
2. Alt 預設會啟動視窗的系統選單；要在 WM_SYSCOMMAND 攔截 SC_KEYMENU 並 return 0 才能徹底抑制。
3. WM_SYSKEYDOWN 要靠 lp & 0x40000000 過濾掉 auto-repeat，否則按住 Alt 會反覆切換。

5. 截圖

四種 view mode 都可由 Z 鍵或 view <name> 指令切換。下列截圖在 1280 × 720、Sponza 室內中央視角拍攝。

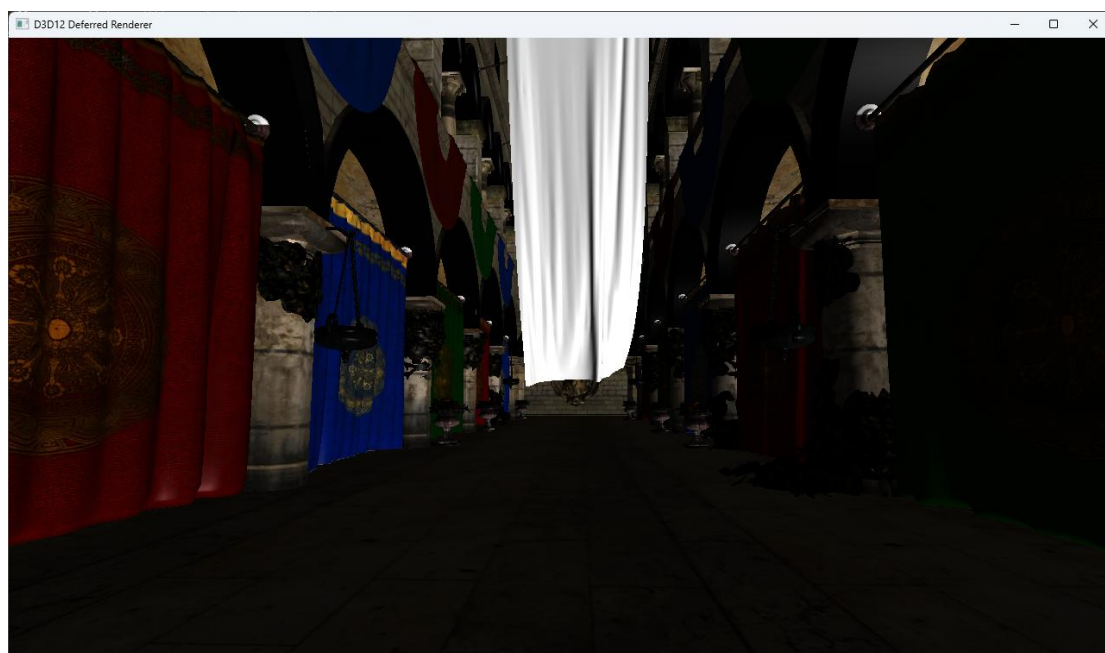


圖 1. Color — 最終 Blinn-Phong 合成結果

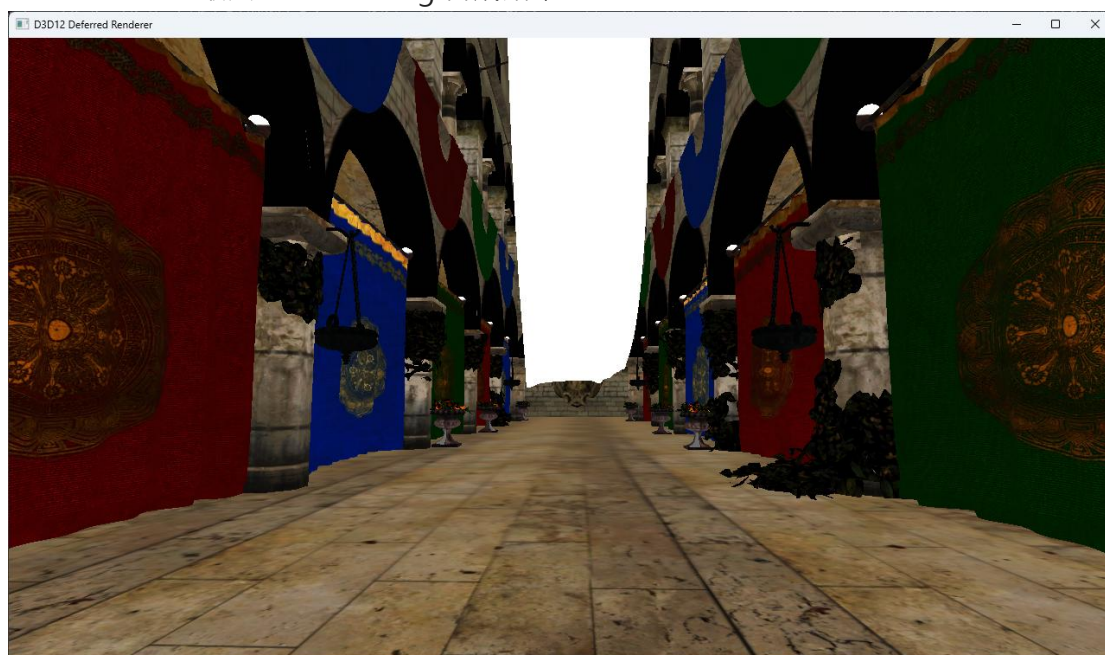


圖 2. Albedo G-buffer

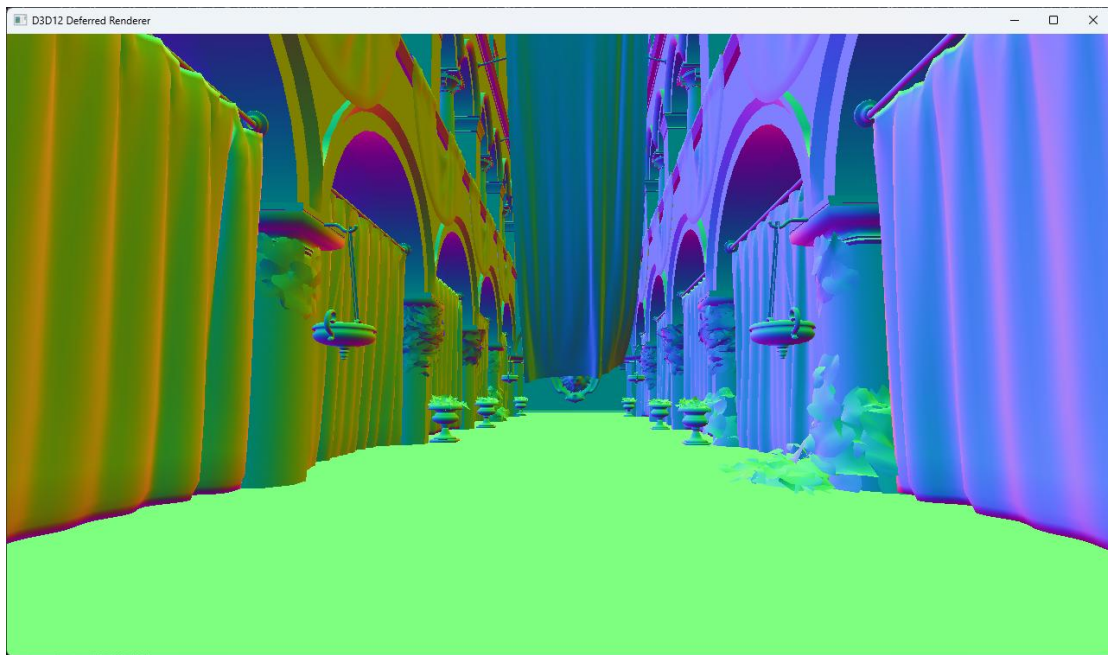


圖 3. Normal G-buffer (重新映射到 $[0, 1]$)

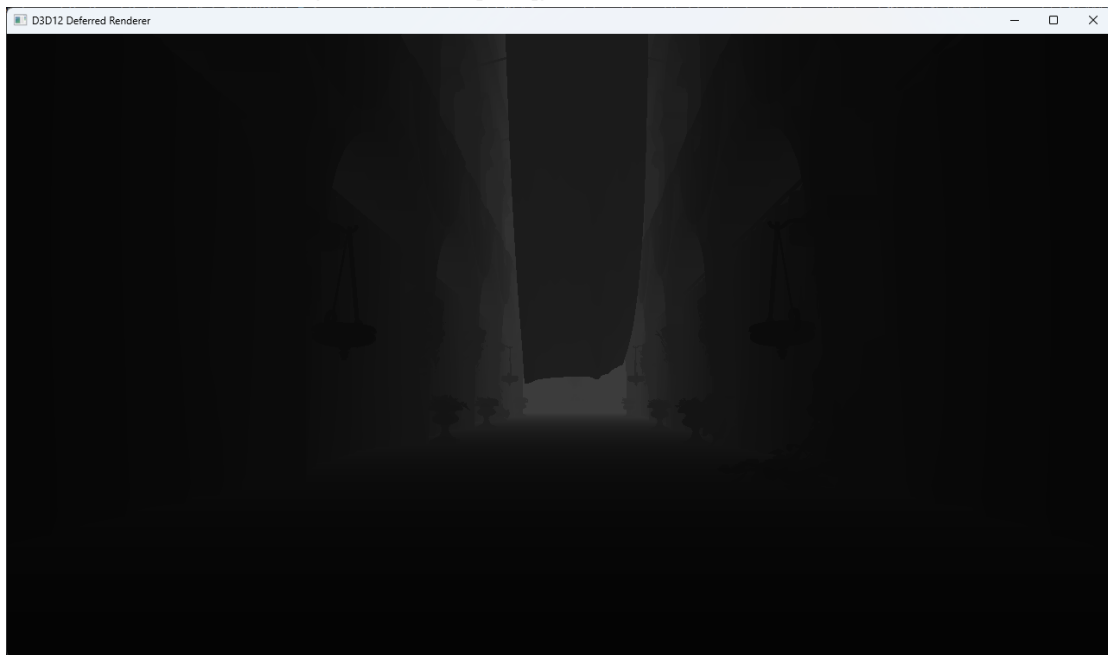


圖 4. Depth G-buffer (依 z_{Near} / z_{Far} 線性化)

6. 編譯與執行

1. 安裝 Visual Studio 2022，勾選「使用 C++ 的桌面開發」(Desktop development with C++) 與 Windows 11 SDK。
2. Clone vcpkg 到 ASCII 路徑並 bootstrap：

```
3. git clone https://github.com/microsoft/vcpkg.git C:\dev\vcpkg  
   C:\dev\vcpkg\bootstrap-vcpkg.bat
```

4. 於專案根目錄安裝相依套件 (buildtrees / packages / install root 都用 ASCII 路徑) :

```
5. C:\dev\vcpkg\vcpkg.exe install ^
6.   --triplet=x64-windows ^
7.   --x-manifest-root=. ^
8.   --x-install-root=C:\vcpkg-cache\installed ^
9.   --x-buildtrees-root=C:\vcpkg-cache\bld ^
   --x-packages-root=C:\vcpkg-cache\pkg
```

10. 用 Visual Studio 2022 開啟 `DeferredRenderer.sln` 並 Build (Debug 或 Release · x64) 。也可命令列 :

```
msbuild DeferredRenderer.sln /m /p:Configuration=Debug
/p:Platform=x64
```

11. 把作業 §5 提供的 `sponza/` 資料夾複製到專案根目錄 (與 `DeferredRenderer.sln` 同層) ; 本份繳交檔不重複附上 Sponza 資產。
12. 從專案根目錄執行 `build/x64/Debug/DeferredRenderer.exe` (或 Release) , 這樣相對路徑 `sponza/sponza.obj` 才能被解析 ; 若用 Visual Studio Debug , 把 Working Directory 設為 `$(SolutionDir)` 。

7. 繳交檔案清單

路徑	說明
<code>DeferredRenderer.sln</code>	Visual Studio Solution
<code>vcpkg.json</code>	Manifest : assimp + directxtk12 + directx-headers
<code>CLAUDE.md</code>	專案備忘 (建置陷阱、規範、操作要求)
<code>Report.docx</code> / <code>Report.pdf</code>	本份報告 (.docx 為原稿 · PDF 由開發者自行匯出)

DeferredRenderer/DeferredRenderer.vcxproj /.filters	VS 專案檔
DeferredRenderer/src/	所有 .cpp / .h 原始碼
DeferredRenderer/shaders/Geometry.hlsl	Geometry Pass Shader
DeferredRenderer/shaders/Lighting.hlsl	Lighting Pass Shader (Full-Screen Triangle)
screenshots/	報告引用的四張截圖 (放在 .docx 內嵌；資 料夾保留供 .md / .html 開啟)